

1-Hour GitHub Project: AgentOps Copilot

A small but portfolio-worthy .NET agent application using Microsoft Foundry + DeepSeek-V3.2-1. It demonstrates a clearly scoped agent, strong instructions, tools, knowledge grounding, persistent memory, guardrails, and structured output.

Implementation target: Build this with Claude Code in approximately 60 minutes. Keep data mocked/local so the repository is easy to run and review.

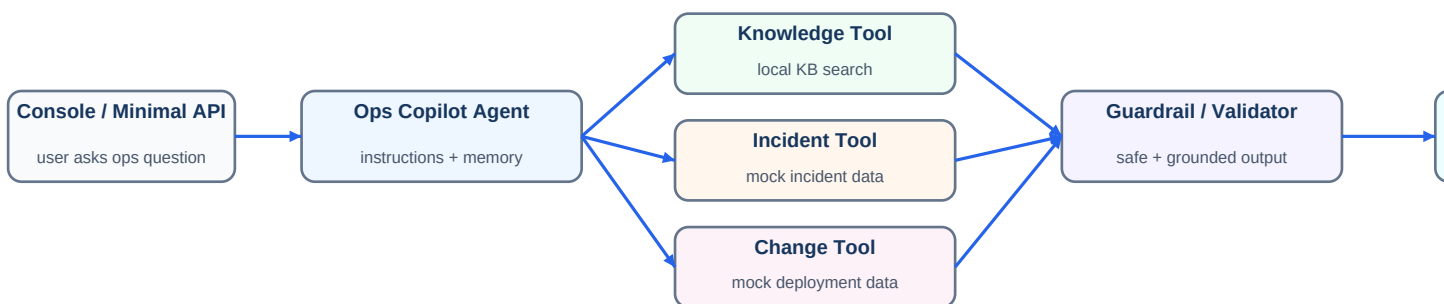
1. What the project is about

Create an **AI Operations Copilot** for a fictional SaaS engineering team. A developer can ask: *"Checkout API is failing after today's deployment - what should I investigate?"* The agent combines internal runbook knowledge with mock incident/change data, remembers conversation context, and returns a grounded troubleshooting plan.

Why this adds value to GitHub

- More than a chatbot: the model decides when to use tools and internal knowledge.
- Shows a realistic enterprise .NET + agentic AI use case.
- Demonstrates instructions, tools, grounding, memory, guardrails, structured output, and Foundry integration in a small repository.

2. Architecture



3. Required agent

Agent: OpsCopilotAgent. It helps engineers investigate operational incidents using supplied knowledge and tool results. It gathers evidence before making claims, explains uncertainty, and produces actionable next steps.

Agent instructions

- You are an internal SaaS operations copilot for engineers.
- For incident-specific questions, inspect available incident/change tools before giving a diagnosis.
- Use the internal knowledge base for runbooks and standard operating procedures.
- Never invent an incident, deployment, metric, owner, or runbook fact.
- If evidence is insufficient, explicitly say what information is missing.
- Never execute destructive/remediation actions; only recommend them.
- Prefer short ordered troubleshooting steps and identify the evidence used.

4. Tools to implement

Tool	Input	Purpose
GetActiveIncidents	serviceName?	Mock active incidents: ID, service, severity, symptoms, start time.
GetRecentChanges	serviceName	Mock recent deployments/configuration changes.
SearchRunbooks	query	Search a small local runbook collection; return relevant snippets + filenames.

All tools may use hard-coded/mock JSON. Do not integrate GitHub, Azure Monitor, Jira, or a real database in the one-hour version.

5. Knowledge / grounding

Create Knowledge/ with 3 short Markdown files. Keep each roughly 10-20 lines. Use simple keyword/scored search; do **not** add a vector database for this version.

- checkout-api-runbook.md - common 5xx causes, dependency checks, rollback criteria.
- database-runbook.md - connection-pool exhaustion, timeout checks, safe escalation steps.
- incident-policy.md - severity definitions, escalation expectations, communication rules.

SearchRunbooks returns the top 1-3 relevant snippets plus filename. This creates a small RAG-like grounding layer while staying inside the one-hour scope.

6. Persistent memory

Persist conversation state to Data/session.json. Save user/assistant messages and reload them at startup. Keep only the most recent ~10 messages or a compact summary so context does not grow indefinitely.

Demo persistence by closing/reopening the app and asking: *"What was the incident ID we were discussing?"*

7. Guardrails

- Keep the Foundry deployment's existing DefaultV2 provider guardrail enabled.
- Application guardrail: reject obviously destructive requests such as deleting production data or disabling security controls; offer safe escalation instead.
- Grounding guardrail: distinguish evidence from hypothesis and never cite a source/tool result that was not actually returned.

8. Structured output

Return a typed result rather than only free-form text:

```
public sealed record OpsAnalysis(
    string Summary,
    string Severity,
    List<string> Evidence,
    List<string> RecommendedActions,
    List<string> Sources,
    bool NeedsHumanEscalation);
```

If the selected DeepSeek client path does not reliably enforce a schema, request JSON and deserialize/validate it in .NET. Fail gracefully if parsing fails.

9. Foundry integration

Use the already working deployment DeepSeek-V3.2-1. Read endpoint/deployment/authentication from configuration, environment variables, or user-secrets.

Never commit a real API key. Do not place secrets in source, README, tests, sample output, or Git history. Provide an example config or documented environment-variable names.

Use the existing working Foundry/OpenAI ChatClient sample as the starting connection code. Wrap model access behind an IAgentService (or equivalent) so the application is not tightly coupled to SDK calls.

10. Suggested structure

```

AgentOpsCopilot/
|-- Program.cs
|-- Agents/OpsCopilotAgent.cs
|-- Tools/
|   |-- IncidentTools.cs
|   |-- ChangeTools.cs
|   |-- RunbookSearchTool.cs
|-- Models/OpsAnalysis.cs
|-- Memory/JsonConversationStore.cs
|-- Guardrails/SafetyValidator.cs
|-- Knowledge/
|   |-- checkout-api-runbook.md
|   |-- database-runbook.md
|   |-- incident-policy.md
|-- Data/
|   |-- incidents.json
|   |-- changes.json
|-- README.md

```

11. Required demo scenario

User: "The Checkout API started returning 500 errors after today's release. What should I do?"

- Search active incidents.
- Inspect recent Checkout API changes.
- Search the Checkout API runbook.
- Produce OpsAnalysis containing evidence, recommended actions, sources, and escalation status.
- A follow-up question demonstrates memory.

12. Definition of done

- Runs locally and calls DeepSeek-V3.2-1 through Microsoft Foundry.
- Agent has explicit instructions and at least three callable tools.
- Local knowledge is searched and sources appear in the final result.
- Conversation history survives application restart.
- Provider + application-level guardrails are documented.
- Response is parsed into OpsAnalysis.
- README contains architecture, setup, sample prompt/output, limitations, and an explanation of each agentic feature.

13. 60-minute plan

Time	Task
0-10 min	Solution/configuration, models, mock JSON, 3 runbook files.
10-25 min	Wrap Foundry ChatClient and implement three tools.
25-35 min	Agent instructions + tool-calling loop.
35-43 min	JSON conversation persistence + restore.
43-50 min	SafetyValidator + OpsAnalysis parsing.
50-60 min	Run demo; polish README with architecture and sample output.

14. Explicit non-goals

- No React/Blazor UI - console or minimal API is enough.
- No vector DB, embeddings pipeline, MCP server, workflow engine, authentication, Docker, or cloud database.
- No real production actions; tools are read-only and use mock/local data.

- Do not over-engineer abstractions merely to increase file count.

15. Instructions to Claude Code

Implement the project end-to-end from this specification. Prefer clear idiomatic .NET code and keep the solution intentionally small. Reuse the user's working Microsoft Foundry connection pattern for DeepSeek-V3.2-1. Before adding SDK-specific APIs, verify they exist in the installed package version rather than inventing methods. Keep secrets outside source control. Build/run the project, fix compilation errors, exercise the required demo scenario, and create a polished README explaining the architecture and agentic features.

Portfolio goal: A reviewer should understand within two minutes that this is a grounded, tool-using, persistent, safety-aware .NET AI agent - not merely a prompt sent to an LLM.